Priority 6 — For Admin:

21.src/pages/admin/AdminDashboard.tsx

```
import React, { useState, useEffect } from 'react';
import { Link } from 'react-router-dom';
import {
  Users, UserCheck, ShieldCheck, Crown,
  Clock, Flag, Heart, CheckCircle,
  ChevronRight, ExternalLink, ThumbsUp, ThumbsDown
} from 'lucide-react';
import toast from 'react-hot-toast';
import { useAuthStore } from '../../store/authStore';
import {
  getAdminStats,
  getAdminUsers,
  getPendingVerifications,
  approveDocument,
  rejectDocument,
  getMessageReports,
  getUnblockRequests
} from '../../lib/actions/adminActions';
import Card from '../../components/ui/Card';
import Spinner from '../../components/ui/Spinner';
import Avatar from '../../components/ui/Avatar';
import Button from '../../components/ui/Button';
import Badge from '../../components/ui/Badge';
import { formatDate } from '../../lib/utils';

export default function AdminDashboard() {
  const { user: adminUser } = useAuthStore();
  const [stats, setStats] = useState<any>(null);
  const [recentUsers, setRecentUsers] = useState<any[]>([]);
  const [pendingDocs, setPendingDocs] = useState<any[]>([]);
  const [pendingMessageReports, setPendingMessageReports] =
useState<any[]>([]);
  const [pendingUnblockRequests, setPendingUnblockRequests] =
useState<any[]>([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
```

```
    fetchDashboardData();
  }, []);

  const fetchDashboardData = async () => {
    try {
      setLoading(true);
      const [statsRes, usersRes, docsRes, reportsRes, unblockRes] = await
Promise.all([
        getAdminStats(),
        getAdminUsers(1, 10),
        getPendingVerifications(),
        getMessageReports(1, 'pending'),
        getUnblockRequests('pending')
      ]);
      setStats(statsRes);
      setRecentUsers(usersRes.users);
      setPendingDocs(docsRes);
      setPendingMessageReports(reportsRes.reports);
      setPendingUnblockRequests(unblockRes);
    } catch (error) {
      console.error('Error fetching admin data:', error);
      toast.error('Failed to load dashboard data');
    } finally {
      setLoading(false);
    }
  };

  const handleApprove = async (docId: string) => {
    if (!adminUser) return;
    try {
      await approveDocument(docId, adminUser.id);
      toast.success('Document approved');
      fetchDashboardData();
    } catch (error) {
      toast.error('Failed to approve document');
    }
  };

  const handleReject = async (docId: string) => {
    if (!adminUser) return;
```

```
    const reason = window.prompt('Enter reason for rejection:');
    if (reason === null) return;
    if (!reason) {
      toast.error('Reason is required for rejection');
      return;
    }
    try {
      await rejectDocument(docId, adminUser.id, reason);
      toast.success('Document rejected');
      fetchDashboardData();
    } catch (error) {
      toast.error('Failed to reject document');
    }
  };

  if (loading && !stats) {
    return (
      <div className="flex items-center justify-center min-h-[60vh]">
        <Spinner size="lg" />
      </div>
    );
  }

  const statCards = [
    { label: "Total Users", value: stats?.totalUsers, icon: <Users
size={24} />, color: "bg-blue-500" },
    { label: "Active Users", value: stats?.activeUsers, icon: <UserCheck
size={24} />, color: "bg-green-500" },
    { label: "Verified Users", value: stats?.verifiedUsers, icon:
<ShieldCheck size={24} />, color: "bg-emerald-500" },
    { label: "Premium Users", value: stats?.premiumUsers, icon: <Crown
size={24} />, color: "bg-yellow-500" },
    { label: "Pending Verifications", value: stats?.pendingDocs, icon:
<Clock size={24} />, color: "bg-orange-500" },
    { label: "Message Reports", value: pendingMessageReports.length, icon:
<Flag size={24} />, color: "bg-red-500" },
    { label: "Unblock Requests", value: pendingUnblockRequests.length,
icon: <ShieldCheck size={24} />, color: "bg-purple-500" },
    { label: "Total Interests", value: stats?.totalInterests, icon: <Heart
size={24} />, color: "bg-pink-500" }
```

```
  ];

  return (
    <div className="space-y-8">
      <div>
        <h1 className="text-2xl font-heading font-bold
text-gray-900">Admin Dashboard</h1>
        <p className="text-gray-500">Welcome back! Here's what's happening
today.</p>
      </div>

      {/* Stat Cards */}
      <div className="grid grid-cols-2 lg:grid-cols-4 gap-4">
        {statCards.map((stat, index) => (
          <Card key={index} className="p-4 border-none shadow-sm
hover:shadow-md transition-shadow">
            <div className="flex items-center gap-4">
              <div className={`w-12 h-12 rounded-2xl ${stat.color} flex
items-center justify-center text-white shadow-lg`}>
                {stat.icon}
              </div>
              <div>
                <p className="text-2xl font-bold
text-gray-900">{stat.value}</p>
                <p className="text-xs font-medium text-gray-500 uppercase
tracking-wider">{stat.label}</p>
              </div>
            </div>
          </Card>
        ))}
      </div>

      <div className="grid grid-cols-1 lg:grid-cols-2 gap-8">
        {/* Recent Users */}
        <Card className="p-6 border-none shadow-sm">
          <div className="flex justify-between items-center mb-6">
            <h2 className="text-lg font-bold text-gray-900">Recent
Users</h2>
            <Link to="/admin/users" className="text-primary text-sm
font-bold flex items-center gap-1 hover:underline">
```

```
              View All <ChevronRight size={16} />
            </Link>
          </div>
          <div className="overflow-x-auto">
            <table className="w-full text-left">
              <thead className="text-xs text-gray-400 uppercase border-b
border-gray-50">
                <tr>
                  <th className="pb-3 font-medium">Profile ID</th>
                  <th className="pb-3 font-medium">Name</th>
                  <th className="pb-3 font-medium">Gender</th>
                  <th className="pb-3 font-medium">Joined</th>
                </tr>
              </thead>
              <tbody className="text-sm">
                {recentUsers.map((u) => (
                  <tr key={u.id} className="border-b border-gray-50
last:border-0 hover:bg-gray-50/50 transition-colors">
                    <td className="py-4 font-mono text-xs text-primary">
                      <Link to={`/admin/users/${u.id}`}
className="hover:underline">{u.profile_id}</Link>
                    </td>
                    <td className="py-4 font-medium text-gray-900">
                      <Link to={`/admin/users/${u.id}`}
className="hover:text-primary transition-colors">
                        {u.first_name} {u.last_name}
                      </Link>
                    </td>
                    <td className="py-4 text-gray-600
capitalize">{u.gender}</td>
                    <td className="py-4
text-gray-500">{formatDate(u.created_at)}</td>
                  </tr>
                ))}
              </tbody>
            </table>
          </div>
        </Card>

        {/* Pending Verifications */}
```

```
        <Card className="p-6 border-none shadow-sm">
          <div className="flex justify-between items-center mb-6">
            <h2 className="text-lg font-bold text-gray-900">Pending
Verifications</h2>
            <Badge variant="warning">{pendingDocs.length} Pending</Badge>
          </div>
          <div className="space-y-4">
            {pendingDocs.length > 0 ? (
              pendingDocs.map((doc) => (
                <div key={doc.id} className="flex items-center gap-4 p-4
bg-gray-50 rounded-2xl group">
                  <Avatar
                    src={doc.profiles.profile_photo_url}
                    fallbackName={`${doc.profiles.first_name}
${doc.profiles.last_name}`}
                    size="md"
                    gender={doc.profiles.gender}
                  />
                  <div className="flex-1 min-w-0">
                    <Link to={`/admin/users/${doc.profiles.id}`}
className="font-bold text-gray-900 truncate hover:text-primary
transition-colors block">
                      {doc.profiles.first_name} {doc.profiles.last_name}
                    </Link>
                    <p className="text-xs text-gray-500 uppercase
tracking-wider">
                      {doc.document_type.replace('_', ' ')}
                    </p>
                  </div>
                  <div className="flex items-center gap-2 opacity-0
group-hover:opacity-100 transition-opacity">
                    <a
                      href={doc.file_url}
                      target="_blank"
                      rel="noopener noreferrer"
                      className="p-2 text-blue-500 hover:bg-blue-100
rounded-full transition-colors"
                      title="View Document"
                    >
                      <ExternalLink size={18} />
```

```
                    </a>
                    <button
                      onClick={() => handleApprove(doc.id)}
                      className="p-2 text-green-500 hover:bg-green-100
rounded-full transition-colors"
                      title="Approve"
                    >
                      <ThumbsUp size={18} />
                    </button>
                    <button
                      onClick={() => handleReject(doc.id)}
                      className="p-2 text-red-500 hover:bg-red-100
rounded-full transition-colors"
                      title="Reject"
                    >
                      <ThumbsDown size={18} />
                    </button>
                  </div>
                </div>
              ))
            ) : (
              <div className="text-center py-10 text-gray-400">
                <ShieldCheck size={48} className="mx-auto mb-2 opacity-20"
/>
                <p>No pending verifications</p>
              </div>
            )}
          </div>
        </Card>
      </div>

      <div className="grid grid-cols-1 lg:grid-cols-2 gap-8">
        {/* Pending Message Reports */}
        <Card className="p-6 border-none shadow-sm">
          <div className="flex justify-between items-center mb-6">
            <h2 className="text-lg font-bold text-gray-900">Pending
Message Reports</h2>
            <Link to="/admin/settings" className="text-primary text-sm
font-bold flex items-center gap-1 hover:underline">
              Manage <ChevronRight size={16} />
```

```
            </Link>
          </div>
          <div className="space-y-4">
            {pendingMessageReports.length > 0 ? (
              pendingMessageReports.slice(0, 5).map((report) => (
                <div key={report.id} className="p-4 bg-red-50 rounded-xl
border border-red-100">
                  <div className="flex justify-between items-start mb-2">
                    <div className="text-sm font-medium text-gray-900">
                      Reported by: <Link
to={`/admin/users/${report.reporter.id}`} className="text-primary
hover:underline">{report.reporter.first_name}</Link>
                    </div>
                    <Badge variant="danger">{report.reason}</Badge>
                  </div>
                  <p className="text-sm text-gray-700 italic bg-white p-2
rounded border border-red-100 mb-2">
                    "{report.message_content}"
                  </p>
                  <div className="text-xs text-gray-500">
                    Reported user: <Link
to={`/admin/users/${report.reported_user.id}`} className="font-medium
hover:underline">{report.reported_user.first_name}
{report.reported_user.last_name}</Link>
                  </div>
                </div>
              ))
            ) : (
              <div className="text-center py-8 text-gray-400">
                <Flag size={32} className="mx-auto mb-2 opacity-20" />
                <p>No pending message reports</p>
              </div>
            )}
          </div>
        </Card>

        {/* Pending Unblock Requests */}
        <Card className="p-6 border-none shadow-sm">
          <div className="flex justify-between items-center mb-6">
```

```
            <h2 className="text-lg font-bold text-gray-900">Pending
Unblock Requests</h2>
            <Link to="/admin/settings" className="text-primary text-sm
font-bold flex items-center gap-1 hover:underline">
              Manage <ChevronRight size={16} />
            </Link>
          </div>
          <div className="space-y-4">
            {pendingUnblockRequests.length > 0 ? (
              pendingUnblockRequests.slice(0, 5).map((req) => (
                <div key={req.id} className="p-4 bg-purple-50 rounded-xl
border border-purple-100">
                  <div className="flex justify-between items-start mb-2">
                    <Link to={`/admin/users/${req.user.id}`}
className="font-bold text-gray-900 hover:text-primary transition-colors">
                      {req.user.first_name} {req.user.last_name}
                    </Link>
                    <span className="text-xs
text-gray-500">{formatDate(req.created_at)}</span>
                  </div>
                  <div className="text-xs text-red-600 font-medium mb-2">
                    Blocked for: {req.user.block_reason || 'Safety policy
violation'}
                  </div>
                  <p className="text-sm text-gray-700 bg-white p-2 rounded
border border-purple-100">
                    "{req.reason}"
                  </p>
                </div>
              ))
            ) : (
              <div className="text-center py-8 text-gray-400">
                <ShieldCheck size={32} className="mx-auto mb-2 opacity-20"
/>
                <p>No pending unblock requests</p>
              </div>
            )}
          </div>
        </Card>
      </div>
```

```
    </div>
  );
}
```

22.src/pages/admin/AdminSettings.tsx

```
import React, { useState, useEffect } from 'react';
import { Link } from 'react-router-dom';
import { Settings, Save, RefreshCw, Plus, Trash2, Edit2, CheckCircle,
XCircle, AlertTriangle } from 'lucide-react';
import toast from 'react-hot-toast';
import { useAuthStore } from '../../store/authStore';
import {
  getSiteSettings, updateSiteSetting,
  getMembershipPlans, updateMembershipPlan,
  getUnblockRequests, handleUnblockRequest, getUnblockRequestDetail,
  getMessageReports, handleMessageReport,
  getAdminSuccessStories, updateStoryApproval,
  getAdminContacts, markContactResolved
} from '../../lib/actions/adminActions';
import Card from '../../components/ui/Card';
import Spinner from '../../components/ui/Spinner';
import Button from '../../components/ui/Button';
import Badge from '../../components/ui/Badge';
import Modal from '../../components/ui/Modal';
import { formatDate } from '../../lib/utils';

export default function AdminSettings() {
  const { user } = useAuthStore();
  const [activeTab, setActiveTab] = useState('general');
  const [loading, setLoading] = useState(true);
  const [saving, setSaving] = useState(false);

  // Settings State
  const [settings, setSettings] = useState<any[]>([]);
  const [editedSettings, setEditedSettings] = useState<Record<string,
any>>({});

  // Plans State
  const [plans, setPlans] = useState<any[]>([]);
```

```
  const [editingPlan, setEditingPlan] = useState<any | null>(null);

  // Unblock Requests State
  const [unblockRequests, setUnblockRequests] = useState<any[]>([]);
  const [selectedRequest, setSelectedRequest] = useState<any |
null>(null);
  const [selectedRequestDetail, setSelectedRequestDetail] = useState<any |
null>(null);
  const [adminNotes, setAdminNotes] = useState('');
  const [loadingRequestDetail, setLoadingRequestDetail] = useState(false);

  // Message Reports State
  const [messageReports, setMessageReports] = useState<any[]>([]);
  const [selectedReport, setSelectedReport] = useState<any | null>(null);
  const [reportAdminNotes, setReportAdminNotes] = useState('');

  // Success Stories State
  const [successStories, setSuccessStories] = useState<any[]>([]);

  // Contact Messages State
  const [contacts, setContacts] = useState<any[]>([]);

  useEffect(() => {
    fetchData();
  }, [activeTab]);

  const fetchData = async () => {
    setLoading(true);
    try {
      if (activeTab === 'general') {
        const data = await getSiteSettings();
        setSettings(data);
        const initialEdits: Record<string, any> = {};
        data.forEach((s: any) => {
          initialEdits[s.setting_key] = s.setting_value;
        });
        setEditedSettings(initialEdits);
      } else if (activeTab === 'plans') {
        const data = await getMembershipPlans();
        setPlans(data);
```

```
      } else if (activeTab === 'unblock') {
        const data = await getUnblockRequests();
        setUnblockRequests(data);
      } else if (activeTab === 'reports') {
        const data = await getMessageReports(1, 'pending');
        setMessageReports(data.reports);
      } else if (activeTab === 'stories') {
        const data = await getAdminSuccessStories();
        setSuccessStories(data.stories);
      } else if (activeTab === 'contacts') {
        const data = await getAdminContacts();
        setContacts(data.contacts);
      }
    } catch (error) {
      toast.error('Failed to fetch data');
    } finally {
      setLoading(false);
    }
  };

  const handleApproveStory = async (storyId: string, approved: boolean) =>
{
    setSaving(true);
    try {
      await updateStoryApproval(storyId, approved);
      toast.success(`Story ${approved ? 'approved' : 'rejected'}
successfully`);
      fetchData();
    } catch (error) {
      toast.error('Failed to update story');
    } finally {
      setSaving(false);
    }
  };

  const handleResolveContact = async (contactId: string) => {
    setSaving(true);
    try {
      await markContactResolved(contactId);
      toast.success('Contact marked as resolved');
```

```
      fetchData();
    } catch (error) {
      toast.error('Failed to resolve contact');
    } finally {
      setSaving(false);
    }
  };

  const handleSaveSettings = async () => {
    if (!user) return;
    setSaving(true);
    try {
      for (const key in editedSettings) {
        const original = settings.find(s => s.setting_key === key);
        if (original && original.setting_value !== editedSettings[key]) {
          await updateSiteSetting(key, editedSettings[key], user.id);
        }
      }
      toast.success('Settings updated successfully');
      fetchData();
    } catch (error) {
      toast.error('Failed to update settings');
    } finally {
      setSaving(false);
    }
  };

  const handleSettingChange = (key: string, value: any) => {
    setEditedSettings(prev => ({ ...prev, [key]: value }));
  };

  const handleSavePlan = async (e: React.FormEvent) => {
    e.preventDefault();
    setSaving(true);
    try {
      await updateMembershipPlan(editingPlan.id, editingPlan);
      toast.success('Plan updated successfully');
      setEditingPlan(null);
      fetchData();
    } catch (error) {
```

```
      toast.error('Failed to update plan');
    } finally {
      setSaving(false);
    }
  };

  const handleProcessUnblockRequest = async (requestId: string, status:
'approved' | 'rejected') => {
    if (!user) return;
    if (!adminNotes && status === 'rejected') {
      toast.error('Please provide notes for rejection');
      return;
    }

    setSaving(true);
    try {
      await handleUnblockRequest(requestId, user.id, status, adminNotes);
      toast.success(`Request ${status} successfully`);
      setSelectedRequest(null);
      setSelectedRequestDetail(null);
      setAdminNotes('');
      fetchData();
    } catch (error) {
      toast.error('Failed to process request');
    } finally {
      setSaving(false);
    }
  };

  const handleSelectRequest = async (req: any) => {
    setSelectedRequest(req);
    setSelectedRequestDetail(null);
    setLoadingRequestDetail(true);
    try {
      const detail = await getUnblockRequestDetail(req.id);
      setSelectedRequestDetail(detail);
    } catch (error) {
      toast.error('Failed to fetch request details');
    } finally {
      setLoadingRequestDetail(false);
```

```
    }
  };

  const handleProcessMessageReport = async (reportId: string, action:
'dismissed' | 'warned' | 'blocked') => {
    if (!user) return;
    setSaving(true);
    try {
      await handleMessageReport(reportId, user.id, action,
reportAdminNotes);
      toast.success(`Report processed: User ${action}`);
      setSelectedReport(null);
      setReportAdminNotes('');
      fetchData();
    } catch (error) {
      toast.error('Failed to process report');
    } finally {
      setSaving(false);
    }
  };

  return (
    <div className="space-y-6 max-w-6xl mx-auto pb-12">
      <div>
        <h1 className="text-2xl font-heading font-bold text-gray-900 flex
items-center gap-2">
          <Settings size={28} className="text-primary" /> System Settings
        </h1>
        <p className="text-gray-500">Manage site configuration, membership
plans, and user requests</p>
      </div>

      {/* Tabs */}
      <div className="flex border-b border-gray-200">
        <button
          className={`px-6 py-3 font-medium text-sm transition-colors
relative ${activeTab === 'general' ? 'text-primary' : 'text-gray-500
hover:text-gray-700'}`}
          onClick={() => setActiveTab('general')}
        >
```

```
          General Settings
          {activeTab === 'general' && <div className="absolute bottom-0
left-0 right-0 h-0.5 bg-primary rounded-t-full" />}
        </button>
        <button
          className={`px-6 py-3 font-medium text-sm transition-colors
relative ${activeTab === 'plans' ? 'text-primary' : 'text-gray-500
hover:text-gray-700'}`}
          onClick={() => setActiveTab('plans')}
        >
          Membership Plans
          {activeTab === 'plans' && <div className="absolute bottom-0
left-0 right-0 h-0.5 bg-primary rounded-t-full" />}
        </button>
        <button
          className={`px-6 py-3 font-medium text-sm transition-colors
relative ${activeTab === 'unblock' ? 'text-primary' : 'text-gray-500
hover:text-gray-700'}`}
          onClick={() => setActiveTab('unblock')}
        >
          Unblock Requests
          {activeTab === 'unblock' && <div className="absolute bottom-0
left-0 right-0 h-0.5 bg-primary rounded-t-full" />}
        </button>
        <button
          className={`px-6 py-3 font-medium text-sm transition-colors
relative ${activeTab === 'reports' ? 'text-primary' : 'text-gray-500
hover:text-gray-700'}`}
          onClick={() => setActiveTab('reports')}
        >
          Message Reports
          {activeTab === 'reports' && <div className="absolute bottom-0
left-0 right-0 h-0.5 bg-primary rounded-t-full" />}
        </button>
        <button
          className={`px-6 py-3 font-medium text-sm transition-colors
relative ${activeTab === 'stories' ? 'text-primary' : 'text-gray-500
hover:text-gray-700'}`}
          onClick={() => setActiveTab('stories')}
        >
```

```
          Success Stories
          {activeTab === 'stories' && <div className="absolute bottom-0
left-0 right-0 h-0.5 bg-primary rounded-t-full" />}
        </button>
        <button
          className={`px-6 py-3 font-medium text-sm transition-colors
relative ${activeTab === 'contacts' ? 'text-primary' : 'text-gray-500
hover:text-gray-700'}`}
          onClick={() => setActiveTab('contacts')}
        >
          Contact Messages
          {activeTab === 'contacts' && <div className="absolute bottom-0
left-0 right-0 h-0.5 bg-primary rounded-t-full" />}
        </button>
      </div>

      {/* Content */}
      <div className="mt-6">
        {loading ? (
          <div className="flex justify-center items-center h-64">
            <Spinner size="lg" />
          </div>
        ) : (
          <>
            {/* General Settings Tab */}
            {activeTab === 'general' && (
              <Card className="p-6">
                <div className="space-y-6">
                  {settings.map((setting) => (
                    <div key={setting.id} className="grid grid-cols-1
md:grid-cols-3 gap-4 border-b border-gray-100 pb-6 last:border-0
last:pb-0">
                      <div className="md:col-span-1">
                        <label className="block text-sm font-bold
text-gray-900">{setting.setting_key.replace(/_/g, '
').toUpperCase()}</label>
                        <p className="text-xs text-gray-500
mt-1">{setting.description}</p>
                      </div>
                      <div className="md:col-span-2">
```

```
                        {setting.setting_type === 'boolean' ? (
                          <div className="flex items-center gap-3">
                            <label className="relative inline-flex
items-center cursor-pointer">
                              <input
                                type="checkbox"
                                className="sr-only peer"

checked={editedSettings[setting.setting_key] === 'true'}
                                onChange={(e) =>
handleSettingChange(setting.setting_key, e.target.checked ? 'true' :
'false')}
                              />
                              <div className="w-11 h-6 bg-gray-200
peer-focus:outline-none peer-focus:ring-4 peer-focus:ring-primary/20
rounded-full peer peer-checked:after:translate-x-full
peer-checked:after:border-white after:content-[''] after:absolute
after:top-[2px] after:left-[2px] after:bg-white after:border-gray-300
after:border after:rounded-full after:h-5 after:w-5 after:transition-all
peer-checked:bg-primary"></div>
                            </label>
                            <span className="text-sm font-medium
text-gray-700">
                              {editedSettings[setting.setting_key] ===
'true' ? 'Enabled' : 'Disabled'}
                            </span>
                          </div>
                        ) : setting.setting_type === 'number' ? (
                          <input
                            type="number"
                            className="w-full max-w-md px-4 py-2 border
border-gray-300 rounded-xl focus:ring-2 focus:ring-primary
focus:border-transparent outline-none transition-all"
                            value={editedSettings[setting.setting_key] ||
''}
                            onChange={(e) =>
handleSettingChange(setting.setting_key, e.target.value)}
                          />
                        ) : (
                          <input
```

```
                            type="text"
                            className="w-full max-w-md px-4 py-2 border
border-gray-300 rounded-xl focus:ring-2 focus:ring-primary
focus:border-transparent outline-none transition-all"
                            value={editedSettings[setting.setting_key] ||
''}
                            onChange={(e) =>
handleSettingChange(setting.setting_key, e.target.value)}
                          />
                        )}
                      </div>
                    </div>
                  ))}

                  <div className="pt-4 flex justify-end">
                    <Button onClick={handleSaveSettings} disabled={saving}
className="min-w-[120px]">
                      {saving ? <Spinner size="sm" /> : <><Save size={18}
className="mr-2" /> Save Settings</>}
                    </Button>
                  </div>
                </div>
              </Card>
            )}

            {/* Membership Plans Tab */}
            {activeTab === 'plans' && (
              <div className="grid grid-cols-1 md:grid-cols-2
lg:grid-cols-3 gap-6">
                {plans.map((plan) => (
                  <Card key={plan.id} className="p-6 flex flex-col h-full
border-2 border-transparent hover:border-primary/20 transition-all">
                    <div className="flex justify-between items-start
mb-4">
                      <Badge variant="primary" className="uppercase
tracking-wider font-bold">{plan.plan_name}</Badge>
                      <button
                        onClick={() => setEditingPlan(plan)}
                        className="p-2 text-gray-400 hover:text-primary
hover:bg-primary/10 rounded-lg transition-colors"
```

```
                      >
                        <Edit2 size={18} />
                      </button>
                    </div>

                    <div className="mb-6">
                      <span className="text-3xl font-bold
text-gray-900">₹{plan.plan_price}</span>
                      <span className="text-gray-500 text-sm"> /
{plan.duration_months} months</span>
                    </div>

                    <div className="flex-1">
                      <p className="text-sm font-bold text-gray-700
mb-3">Features:</p>
                      <ul className="space-y-2">
                        {plan.features?.map((feature: string, idx: number)
=> (
                          <li key={idx} className="flex items-start gap-2
text-sm text-gray-600">
                            <CheckCircle size={16}
className="text-green-500 mt-0.5 flex-shrink-0" />
                            <span>{feature}</span>
                          </li>
                        ))}
                      </ul>
                    </div>

                    <div className="mt-6 pt-4 border-t border-gray-100
flex justify-between items-center text-xs text-gray-400">
                      <span>ID: {plan.id.substring(0, 8)}</span>
                      <span>{plan.is_active ? 'Active' :
'Inactive'}</span>
                    </div>
                  </Card>
                ))}
              </div>
            )}

            {/* Unblock Requests Tab */}
```

```
            {activeTab === 'unblock' && (
              <Card className="overflow-hidden border-none shadow-sm">
                <div className="overflow-x-auto">
                  <table className="w-full text-left">
                    <thead className="bg-gray-50 text-xs uppercase
tracking-wider text-gray-500 font-bold border-b border-gray-100">
                      <tr>
                        <th className="px-6 py-4">User</th>
                        <th className="px-6 py-4">Reason Provided</th>
                        <th className="px-6 py-4">Date Submitted</th>
                        <th className="px-6 py-4">Status</th>
                        <th className="px-6 py-4 text-right">Actions</th>
                      </tr>
                    </thead>
                    <tbody className="divide-y divide-gray-50">
                      {unblockRequests.length > 0 ? (
                        unblockRequests.map((req) => (
                          <tr key={req.id} className="hover:bg-gray-50/50
transition-colors">
                            <td className="px-6 py-4">
                              <div className="text-sm">
                                <Link to={`/admin/users/${req.user_id}`}
className="font-bold text-gray-900 hover:text-primary transition-colors
block">
                                  {req.user.first_name}
{req.user.last_name}
                                </Link>
                                <p className="text-xs text-primary
font-mono">{req.user.profile_id}</p>
                              </div>
                            </td>
                            <td className="px-6 py-4">
                              <p className="text-sm text-gray-600 truncate
max-w-[300px]">{req.reason}</p>
                            </td>
                            <td className="px-6 py-4 text-sm
text-gray-500">{formatDate(req.created_at)}</td>
                            <td className="px-6 py-4">
                              <Badge
                                variant={
```

```
                                req.status === 'pending' ? 'warning' :
                                req.status === 'approved' ? 'success' :
'danger'
                              }
                            >
                              {req.status}
                            </Badge>
                          </td>
                          <td className="px-6 py-4 text-right">
                            <Button
                              variant="outline"
                              size="sm"
                              onClick={() => handleSelectRequest(req)}
                              className="rounded-full"
                            >
                              Review
                            </Button>
                          </td>
                        </tr>
                      ))
                    ) : (
                      <tr>
                        <td colSpan={5} className="px-6 py-12
text-center text-gray-400">
                          No unblock requests found
                        </td>
                      </tr>
                    )}
                  </tbody>
                </table>
              </div>
            </Card>
          )}
          {/* Message Reports Tab */}
          {activeTab === 'reports' && (
            <Card className="overflow-hidden border-none shadow-sm">
              <div className="overflow-x-auto">
                <table className="w-full text-left">
                  <thead className="bg-gray-50 text-xs uppercase
tracking-wider text-gray-500 font-bold border-b border-gray-100">
```

```
                    <tr>
                      <th className="px-6 py-4">Reporter</th>
                      <th className="px-6 py-4">Reported User</th>
                      <th className="px-6 py-4">Reason</th>
                      <th className="px-6 py-4">Date</th>
                      <th className="px-6 py-4 text-right">Actions</th>
                    </tr>
                  </thead>
                  <tbody className="divide-y divide-gray-50">
                    {messageReports.length > 0 ? (
                      messageReports.map((report) => (
                        <tr key={report.id}
className="hover:bg-gray-50/50 transition-colors">
                          <td className="px-6 py-4">
                            <div className="text-sm">
                              <Link
to={`/admin/users/${report.reporter_id}`} className="font-bold
text-gray-900 hover:text-primary transition-colors block">
                                {report.reporter.first_name}
{report.reporter.last_name}
                              </Link>
                            </div>
                          </td>
                          <td className="px-6 py-4">
                            <div className="text-sm">
                              <Link
to={`/admin/users/${report.reported_user_id}`} className="font-bold
text-gray-900 hover:text-primary transition-colors block">
                                {report.reported_user.first_name}
{report.reported_user.last_name}
                              </Link>
                            </div>
                          </td>
                          <td className="px-6 py-4">
                            <Badge
variant="danger">{report.reason}</Badge>
                          </td>
                          <td className="px-6 py-4 text-sm
text-gray-500">{formatDate(report.created_at)}</td>
                          <td className="px-6 py-4 text-right">
```

```
                            <Button
                              variant="outline"
                              size="sm"
                              onClick={() => setSelectedReport(report)}
                              className="rounded-full"
                            >
                              Review
                            </Button>
                          </td>
                        </tr>
                      ))
                    ) : (
                      <tr>
                        <td colSpan={5} className="px-6 py-12
text-center text-gray-400">
                          No pending message reports
                        </td>
                      </tr>
                    )}
                  </tbody>
                </table>
              </div>
            </Card>
          )}

          {/* Success Stories Tab */}
          {activeTab === 'stories' && (
            <div className="grid grid-cols-1 md:grid-cols-2 gap-6">
              {successStories.length > 0 ? (
                successStories.map((story) => (
                  <Card key={story.id} className="p-6 flex flex-col
gap-4">
                    <div className="flex justify-between items-start">
                      <div className="flex items-center gap-3">
                        <div className="w-10 h-10 rounded-full
bg-primary/10 flex items-center justify-center text-primary font-bold">
                          {story.user_id.substring(0, 2).toUpperCase()}
                        </div>
                        <div>
```

```
                            <p className="text-sm font-bold
text-gray-900">Story by User ID: {story.user_id.substring(0, 8)}</p>
                            <p className="text-xs
text-gray-500">{formatDate(story.created_at)}</p>
                          </div>
                        </div>
                        <Badge variant={story.is_approved ? 'success' :
'warning'}>
                          {story.is_approved ? 'Approved' : 'Pending'}
                        </Badge>
                      </div>

                      <div className="bg-gray-50 p-4 rounded-xl text-sm
text-gray-700 italic border border-gray-100">
                        "{story.story_content}"
                      </div>

                      <div className="flex justify-end gap-2 mt-auto pt-4
border-t border-gray-100">
                        {!story.is_approved ? (
                          <Button
                            size="sm"
                            variant="primary"
                            onClick={() => handleApproveStory(story.id,
true)}
                            disabled={saving}
                          >
                            Approve
                          </Button>
                        ) : (
                          <Button
                            size="sm"
                            variant="outline"
                            className="text-red-600 border-red-200
hover:bg-red-50"
                            onClick={() => handleApproveStory(story.id,
false)}
                            disabled={saving}
                          >
                            Reject
```

```
                        </Button>
                      )}
                    </div>
                  </Card>
                ))
              ) : (
                <div className="col-span-full py-12 text-center
text-gray-400 bg-white rounded-3xl border-2 border-dashed
border-gray-100">
                  No success stories found
                </div>
              )}
            </div>
          )}

          {/* Contact Messages Tab */}
          {activeTab === 'contacts' && (
            <Card className="overflow-hidden border-none shadow-sm">
              <div className="overflow-x-auto">
                <table className="w-full text-left">
                  <thead className="bg-gray-50 text-xs uppercase
tracking-wider text-gray-500 font-bold border-b border-gray-100">
                    <tr>
                      <th className="px-6 py-4">Name</th>
                      <th className="px-6 py-4">Subject</th>
                      <th className="px-6 py-4">Message</th>
                      <th className="px-6 py-4">Date</th>
                      <th className="px-6 py-4 text-right">Actions</th>
                    </tr>
                  </thead>
                  <tbody className="divide-y divide-gray-50">
                    {contacts.length > 0 ? (
                      contacts.map((contact) => (
                        <tr key={contact.id}
className={`hover:bg-gray-50/50 transition-colors ${contact.is_resolved ?
'opacity-60' : ''}`}>
                          <td className="px-6 py-4">
                            <div className="text-sm">
                              <p className="font-bold
text-gray-900">{contact.name}</p>
```

```
                                <p className="text-xs
text-gray-500">{contact.email}</p>
                              </div>
                            </td>
                            <td className="px-6 py-4">
                              <p className="text-sm font-medium
text-gray-700">{contact.subject}</p>
                            </td>
                            <td className="px-6 py-4">
                              <p className="text-sm text-gray-600 truncate
max-w-[200px]" title={contact.message}>{contact.message}</p>
                            </td>
                            <td className="px-6 py-4 text-sm
text-gray-500">{formatDate(contact.created_at)}</td>
                            <td className="px-6 py-4 text-right">
                              {!contact.is_resolved ? (
                                <Button
                                  variant="outline"
                                  size="sm"
                                  onClick={() =>
handleResolveContact(contact.id)}
                                  className="rounded-full text-green-600
border-green-200 hover:bg-green-50"
                                >
                                  Resolve
                                </Button>
                              ) : (
                                <Badge variant="success">Resolved</Badge>
                              )}
                            </td>
                          </tr>
                        ))
                      ) : (
                        <tr>
                          <td colSpan={5} className="px-6 py-12
text-center text-gray-400">
                            No contact messages found
                          </td>
                        </tr>
                      )}
```

```
                  </tbody>
                </table>
              </div>
            </Card>
          )}
        </>
      )}
    </div>

    {/* Edit Plan Modal */}
    <Modal
      isOpen={!!editingPlan}
      onClose={() => setEditingPlan(null)}
      title={`Edit Plan: ${editingPlan?.plan_name}`}
      size="md"
    >
      {editingPlan && (
        <form onSubmit={handleSavePlan} className="space-y-4">
          <div>
            <label className="block text-sm font-medium text-gray-700
mb-1">Plan Name</label>
            <input
              type="text"
              className="w-full px-4 py-2 border border-gray-300
rounded-xl focus:ring-2 focus:ring-primary focus:border-transparent
outline-none transition-all"
              value={editingPlan.plan_name}
              onChange={(e) => setEditingPlan({...editingPlan,
plan_name: e.target.value})}
              required
            />
          </div>
          <div className="grid grid-cols-2 gap-4">
            <div>
              <label className="block text-sm font-medium text-gray-700
mb-1">Price (₹)</label>
              <input
                type="number"
```

```
                  className="w-full px-4 py-2 border border-gray-300
rounded-xl focus:ring-2 focus:ring-primary focus:border-transparent
outline-none transition-all"
                  value={editingPlan.plan_price}
                  onChange={(e) => setEditingPlan({...editingPlan,
plan_price: e.target.value})}
                  required
                />
              </div>
              <div>
                <label className="block text-sm font-medium text-gray-700
mb-1">Duration (Months)</label>
                <input
                  type="number"
                  className="w-full px-4 py-2 border border-gray-300
rounded-xl focus:ring-2 focus:ring-primary focus:border-transparent
outline-none transition-all"
                  value={editingPlan.duration_months}
                  onChange={(e) => setEditingPlan({...editingPlan,
duration_months: e.target.value})}
                  required
                />
              </div>
            </div>

            <div>
              <label className="block text-sm font-medium text-gray-700
mb-1">Features (JSON Array)</label>
              <textarea
                className="w-full px-4 py-2 border border-gray-300
rounded-xl focus:ring-2 focus:ring-primary focus:border-transparent
outline-none transition-all font-mono text-sm"
                rows={5}
                value={JSON.stringify(editingPlan.features, null, 2)}
                onChange={(e) => {
                  try {
                    const parsed = JSON.parse(e.target.value);
                    setEditingPlan({...editingPlan, features: parsed});
                  } catch (err) {
```

```
                    // Don't update if invalid JSON, but maybe show a
warning
                  }
                }}
              />
              <p className="text-xs text-gray-500 mt-1">Must be valid JSON
format.</p>
            </div>

            <div className="flex items-center gap-2 mt-4">
              <input
                type="checkbox"
                id="isActive"
                checked={editingPlan.is_active}
                onChange={(e) => setEditingPlan({...editingPlan,
is_active: e.target.checked})}
                className="text-primary focus:ring-primary rounded"
              />
              <label htmlFor="isActive" className="text-sm font-medium
text-gray-700">Plan is Active</label>
            </div>

            <div className="flex justify-end gap-3 pt-4">
              <Button type="button" variant="outline" onClick={() =>
setEditingPlan(null)}>Cancel</Button>
              <Button type="submit" variant="primary"
disabled={saving}>Save Changes</Button>
            </div>
          </form>
        )}
      </Modal>

      {/* Review Unblock Request Modal */}
      <Modal
        isOpen={!!selectedRequest}
        onClose={() => {
          setSelectedRequest(null);
          setSelectedRequestDetail(null);
        }}
        title="Review Unblock Request"
```

```
        size="lg"
      >
        {selectedRequest && (
          <div className="space-y-6">
            <div className="bg-gray-50 p-4 rounded-xl border
border-gray-100">
              <div className="flex justify-between items-start mb-2">
                <div>
                  <p className="text-xs text-gray-400 uppercase
font-bold">User</p>
                  <Link to={`/admin/users/${selectedRequest.user_id}`}
className="font-bold text-gray-900 hover:text-primary transition-colors
block">
                    {selectedRequest.user.first_name}
{selectedRequest.user.last_name}
                  </Link>
                </div>
                <Badge variant={selectedRequest.status === 'pending' ?
'warning' : selectedRequest.status === 'approved' ? 'success' : 'danger'}>
                  {selectedRequest.status}
                </Badge>
              </div>
              <p className="text-xs text-primary font-mono
mb-4">{selectedRequest.user.profile_id}</p>

              <p className="text-xs text-gray-400 uppercase font-bold
mb-1">User's Explanation</p>
              <div className="bg-white p-3 rounded-lg border
border-gray-200 text-sm text-gray-700 italic">
                "{selectedRequest.reason}"
              </div>
            </div>

            {loadingRequestDetail ? (
              <div className="flex justify-center py-8">
                <Spinner size="md" />
              </div>
            ) : selectedRequestDetail && (
              <div className="bg-red-50 p-4 rounded-xl border
border-red-100">
```

```
                <h4 className="font-bold text-red-800 mb-3 flex
items-center gap-2">
                  <AlertTriangle size={18} /> Violation Context
                </h4>
                <div className="space-y-3">
                  <div>
                    <p className="text-xs text-red-600 uppercase font-bold
mb-1">Block Reason</p>
                    <p className="text-sm font-medium
text-red-900">{selectedRequestDetail.user.block_reason || 'Unknown'}</p>
                  </div>

                  {selectedRequestDetail.warnings?.violation_messages &&
selectedRequestDetail.warnings.violation_messages.length > 0 && (
                    <div>
                      <p className="text-xs text-red-600 uppercase
font-bold mb-2">Detected Violations (Messages)</p>
                      <div className="space-y-2 max-h-48 overflow-y-auto
pr-2">

{selectedRequestDetail.warnings.violation_messages.map((vm: any, idx:
number) => (
                          <div key={idx} className="bg-white p-3 rounded
border border-red-200 text-sm">
                            <p className="text-gray-800 font-mono
mb-1">{vm.message}</p>
                            <div className="flex justify-between text-xs
text-gray-500">
                              <span>{formatDate(vm.timestamp)}</span>
                              {vm.chat_with && <span>Chat with: <Link
to={`/admin/users/${vm.chat_with}`} className="text-blue-600
hover:underline">{vm.chat_with.substring(0,8)}...</Link></span>}
                            </div>
                          </div>
                        ))}
                      </div>
                    </div>
                  )}
                </div>
              </div>
```

```
            )}

            {selectedRequest.status === 'pending' ? (
              <div className="space-y-4">
                <div>
                  <label className="block text-sm font-medium
text-gray-700 mb-1">Admin Notes (Required for rejection)</label>
                  <textarea
                    className="w-full px-4 py-2 border border-gray-300
rounded-xl focus:ring-2 focus:ring-primary focus:border-transparent
outline-none transition-all resize-none"
                    rows={3}
                    placeholder="Add notes about this decision..."
                    value={adminNotes}
                    onChange={(e) => setAdminNotes(e.target.value)}
                  ></textarea>
                </div>

                <div className="flex gap-3 pt-2">
                  <Button
                    variant="outline"
                    className="flex-1 text-red-600 border-red-200
hover:bg-red-50"
                    onClick={() =>
handleProcessUnblockRequest(selectedRequest.id, 'rejected')}
                    disabled={saving}
                  >
                    <XCircle size={18} className="mr-2" /> Reject
                  </Button>
                  <Button
                    variant="primary"
                    className="flex-1 bg-green-600 hover:bg-green-700"
                    onClick={() =>
handleProcessUnblockRequest(selectedRequest.id, 'approved')}
                    disabled={saving}
                  >
                    <CheckCircle size={18} className="mr-2" /> Approve &
Unblock
                  </Button>
                </div>
```

```
              </div>
            ) : (
              <div className="bg-blue-50 p-4 rounded-xl border
border-blue-100">
                <p className="text-sm font-bold text-blue-800
mb-1">Request Processed</p>
                <p className="text-sm text-blue-600">This request was
{selectedRequest.status} on {formatDate(selectedRequest.reviewed_at)}.</p>
                {selectedRequest.admin_notes && (
                  <div className="mt-3 pt-3 border-t border-blue-200">
                    <p className="text-xs font-bold text-blue-800
uppercase mb-1">Admin Notes</p>
                    <p className="text-sm
text-blue-700">{selectedRequest.admin_notes}</p>
                  </div>
                )}
              </div>
            )}
          </div>
        )}
      </Modal>
      {/* Review Message Report Modal */}
      <Modal
        isOpen={!!selectedReport}
        onClose={() => setSelectedReport(null)}
        title="Review Message Report"
        size="md"
      >
        {selectedReport && (
          <div className="space-y-6">
            <div className="grid grid-cols-2 gap-4">
              <div className="bg-gray-50 p-4 rounded-xl border
border-gray-100">
                <p className="text-xs text-gray-400 uppercase font-bold
mb-1">Reporter</p>
                <Link to={`/admin/users/${selectedReport.reporter_id}`}
className="font-bold text-gray-900 hover:text-primary transition-colors
block">
                  {selectedReport.reporter.first_name}
{selectedReport.reporter.last_name}
```

```
                </Link>
                <p className="text-xs text-primary
font-mono">{selectedReport.reporter.profile_id}</p>
              </div>
              <div className="bg-red-50 p-4 rounded-xl border
border-red-100">
                <p className="text-xs text-red-400 uppercase font-bold
mb-1">Reported User</p>
                <Link
to={`/admin/users/${selectedReport.reported_user_id}`}
className="font-bold text-red-900 hover:text-red-700 transition-colors
block">
                  {selectedReport.reported_user.first_name}
{selectedReport.reported_user.last_name}
                </Link>
                <p className="text-xs text-red-500
font-mono">{selectedReport.reported_user.profile_id}</p>
              </div>
            </div>

            <div>
              <p className="text-xs text-gray-400 uppercase font-bold
mb-1">Report Reason</p>
              <Badge variant="danger">{selectedReport.reason}</Badge>
            </div>

            <div>
              <p className="text-xs text-gray-400 uppercase font-bold
mb-1">Reported Message</p>
              <div className="bg-white p-4 rounded-xl border
border-gray-200 text-sm text-gray-800 italic shadow-inner
whitespace-pre-wrap">
                "{selectedReport.message_content}"
              </div>
            </div>

            <div className="space-y-4">
              <div>
                <label className="block text-sm font-medium text-gray-700
mb-1">Admin Notes (Optional)</label>
```

```
                <textarea
                  className="w-full px-4 py-2 border border-gray-300
rounded-xl focus:ring-2 focus:ring-primary focus:border-transparent
outline-none transition-all resize-none"
                  rows={2}
                  placeholder="Add notes about this action..."
                  value={reportAdminNotes}
                  onChange={(e) => setReportAdminNotes(e.target.value)}
                ></textarea>
              </div>

              <div className="flex flex-col gap-2 pt-2">
                <Button
                  variant="outline"
                  className="w-full"
                  onClick={() =>
handleProcessMessageReport(selectedReport.id, 'dismissed')}
                  disabled={saving}
                >
                  <CheckCircle size={18} className="mr-2" /> Dismiss
Report (No Action)
                </Button>
                <Button
                  variant="outline"
                  className="w-full border-orange-200 text-orange-600
hover:bg-orange-50"
                  onClick={() =>
handleProcessMessageReport(selectedReport.id, 'warned')}
                  disabled={saving}
                >
                  <AlertTriangle size={18} className="mr-2" /> Warn User
                </Button>
                <Button
                  variant="danger"
                  className="w-full"
                  onClick={() =>
handleProcessMessageReport(selectedReport.id, 'blocked')}
                  disabled={saving}
                >
```

```
                  <XCircle size={18} className="mr-2" /> Block User
(Permanent)
                </Button>
              </div>
            </div>
          </div>
        )}
      </Modal>
    </div>
  );
}
```

23.src/pages/admin/AdminUsers.tsx

```
import React, { useState, useEffect } from 'react';
import { Link } from 'react-router-dom';
import {
  Search, Filter, MoreVertical,
  Shield, ShieldOff, UserCheck, UserX,
  Star, StarOff, ExternalLink, ChevronLeft, ChevronRight, Plus, X
} from 'lucide-react';
import toast from 'react-hot-toast';
import { getAdminUsers, updateUserField, adminAddProfile } from
'../../lib/actions/adminActions';
import Card from '../../components/ui/Card';
import Spinner from '../../components/ui/Spinner';
import Avatar from '../../components/ui/Avatar';
import Button from '../../components/ui/Button';
import Badge from '../../components/ui/Badge';
import Input from '../../components/ui/Input';
import Select from '../../components/ui/Select';
import { formatDate } from '../../lib/utils';

export default function AdminUsers() {
  const [users, setUsers] = useState<any[]>([]);
  const [totalCount, setTotalCount] = useState(0);
  const [loading, setLoading] = useState(true);
  const [page, setPage] = useState(1);
  const [filters, setFilters] = useState({
```

```
    search: '',
    gender: 'all',
    verified: 'all',
    active: 'all',
    premium: 'all',
    blocked: 'all'
  });
  const [activeMenu, setActiveMenu] = useState<string | null>(null);
  const [showAddModal, setShowAddModal] = useState(false);
  const [newProfile, setNewProfile] = useState({
    first_name: '', last_name: '', gender: 'male', date_of_birth: '',
    religion: '', caste: '', phone: '', profile_for: 'myself'
  });
  const [addingProfile, setAddingProfile] = useState(false);

  const limit = 20;

  useEffect(() => {
    fetchUsers();
  }, [page, filters]);

  const fetchUsers = async () => {
    try {
      setLoading(true);
      const res = await getAdminUsers(page, limit, filters);
      setUsers(res.users);
      setTotalCount(res.totalCount);
    } catch (error) {
      toast.error('Failed to fetch users');
    } finally {
      setLoading(false);
    }
  };

  const handleToggleField = async (userId: string, field: string,
currentValue: boolean) => {
    try {
      await updateUserField(userId, field, !currentValue);
      toast.success(`User ${field.replace('is_', '')} status updated`);
      fetchUsers();
```

```
      setActiveMenu(null);
    } catch (error) {
      toast.error('Failed to update user');
    }
  };

  const handleAddProfile = async (e: React.FormEvent) => {
    e.preventDefault();
    setAddingProfile(true);
    try {
      await adminAddProfile(newProfile);
      toast.success('Profile added successfully');
      setShowAddModal(false);
      fetchUsers();
      setNewProfile({
        first_name: '', last_name: '', gender: 'male', date_of_birth: '',
        religion: '', caste: '', phone: '', profile_for: 'myself'
      });
    } catch (error) {
      console.error('Error adding profile:', error);
      toast.error('Failed to add profile');
    } finally {
      setAddingProfile(false);
    }
  };

  const totalPages = Math.ceil(totalCount / limit);

  return (
    <div className="space-y-6">
      <div className="flex flex-col md:flex-row justify-between
items-start md:items-center gap-4">
        <div>
          <h1 className="text-2xl font-heading font-bold
text-gray-900">User Management</h1>
          <p className="text-gray-500">Manage and monitor all registered
profiles</p>
        </div>
        <div className="flex items-center gap-4">
```

```
          <div className="flex items-center gap-2 bg-white p-1 rounded-xl
shadow-sm border border-gray-100">
            <Badge variant="primary" className="px-4 py-1.5">{totalCount}
Total Users</Badge>
          </div>
          <Button onClick={() => setShowAddModal(true)} className="flex
items-center gap-2">
            <Plus size={18} /> Add Profile
          </Button>
        </div>
      </div>

      {/* Filters */}
      <Card className="p-4 border-none shadow-sm space-y-4">
        <div className="grid grid-cols-1 md:grid-cols-6 gap-4">
          <div className="md:col-span-2">
            <Input
              placeholder="Search by name, email or profile ID..."
              value={filters.search}
              onChange={(e) => setFilters({ ...filters, search:
e.target.value })}
              icon={<Search size={18} />}
            />
          </div>
          <Select
            value={filters.gender}
            onChange={(e) => setFilters({ ...filters, gender:
e.target.value })}
            options={[
              { value: 'all', label: 'All Genders' },
              { value: 'male', label: 'Male' },
              { value: 'female', label: 'Female' }
            ]}
          />
          <Select
            value={filters.verified}
            onChange={(e) => setFilters({ ...filters, verified:
e.target.value })}
            options={[
              { value: 'all', label: 'All Verification' },
```

```
              { value: 'yes', label: 'Verified Only' },
              { value: 'no', label: 'Unverified Only' }
            ]}
          />
          <Select
            value={filters.premium}
            onChange={(e) => setFilters({ ...filters, premium:
e.target.value })}
            options={[
              { value: 'all', label: 'All Plans' },
              { value: 'yes', label: 'Premium Only' },
              { value: 'no', label: 'Free Only' }
            ]}
          />
          <Select
            value={filters.blocked}
            onChange={(e) => setFilters({ ...filters, blocked:
e.target.value })}
            options={[
              { value: 'all', label: 'All Status' },
              { value: 'no', label: 'Not Blocked' },
              { value: 'temp', label: 'Temp Blocked' },
              { value: 'permanent', label: 'Perm Blocked' }
            ]}
          />
        </div>
      </Card>

      {/* Users Table */}
      <Card className="overflow-hidden border-none shadow-sm">
        <div className="overflow-x-auto">
          <table className="w-full text-left">
            <thead className="bg-gray-50 text-xs uppercase tracking-wider
text-gray-500 font-bold border-b border-gray-100">
              <tr>
                <th className="px-6 py-4">User</th>
                <th className="px-6 py-4">Profile ID</th>
                <th className="px-6 py-4">Gender</th>
                <th className="px-6 py-4">Joined</th>
                <th className="px-6 py-4">Status</th>
```

```
                <th className="px-6 py-4">Verified</th>
                <th className="px-6 py-4">Premium</th>
                <th className="px-6 py-4 text-right">Actions</th>
              </tr>
            </thead>
            <tbody className="divide-y divide-gray-50">
              {loading ? (
                <tr>
                  <td colSpan={8} className="px-6 py-12 text-center">
                    <Spinner size="md" />
                  </td>
                </tr>
              ) : users.length > 0 ? (
                users.map((u) => (
                  <tr key={u.id} className="hover:bg-gray-50/50
transition-colors">
                    <td className="px-6 py-4">
                      <div className="flex items-center gap-3">
                        <Avatar src={u.profile_photo_url}
fallbackName={`${u.first_name} ${u.last_name}`} size="sm"
gender={u.gender} />
                        <div className="min-w-0">
                          <Link to={`/admin/users/${u.id}`}
className="font-bold text-gray-900 truncate hover:text-primary
transition-colors block">
                            {u.first_name} {u.last_name}
                          </Link>
                          <p className="text-xs text-gray-400
truncate">{u.email}</p>
                        </div>
                      </div>
                    </td>
                    <td className="px-6 py-4 font-mono text-xs
text-primary font-bold">
                      <Link to={`/admin/users/${u.id}`}
className="hover:underline">{u.profile_id}</Link>
                    </td>
                    <td className="px-6 py-4 text-sm text-gray-600
capitalize">{u.gender}</td>
```

```
                    <td className="px-6 py-4 text-sm
text-gray-500">{formatDate(u.created_at)}</td>
                    <td className="px-6 py-4">
                      {u.is_permanently_blocked ? (
                        <Badge variant="danger">Perm Blocked</Badge>
                      ) : u.blocked_until && new Date(u.blocked_until) >
new Date() ? (
                        <Badge variant="warning">Temp Blocked</Badge>
                      ) : u.is_active ? (
                        <Badge variant="success">Active</Badge>
                      ) : (
                        <Badge variant="danger">Inactive</Badge>
                      )}
                    </td>
                    <td className="px-6 py-4">
                      {u.is_verified ? (
                        <div className="w-6 h-6 rounded-full
bg-emerald-100 flex items-center justify-center text-emerald-600">
                          <Shield size={14} />
                        </div>
                      ) : (
                        <div className="w-6 h-6 rounded-full bg-gray-100
flex items-center justify-center text-gray-400">
                          <ShieldOff size={14} />
                        </div>
                      )}
                    </td>
                    <td className="px-6 py-4">
                      {u.is_premium ? (
                        <div className="w-6 h-6 rounded-full bg-yellow-100
flex items-center justify-center text-yellow-600">
                          <Star size={14} />
                        </div>
                      ) : (
                        <div className="w-6 h-6 rounded-full bg-gray-100
flex items-center justify-center text-gray-400">
                          <StarOff size={14} />
                        </div>
                      )}
                    </td>
```

```
                    <td className="px-6 py-4 text-right relative">
                      <button
                        onClick={() => setActiveMenu(activeMenu === u.id ?
null : u.id)}
                        className="p-2 hover:bg-gray-100 rounded-full
transition-colors"
                      >
                        <MoreVertical size={18} />
                      </button>

                      {activeMenu === u.id && (
                        <div className="absolute right-6 top-12 w-48
bg-white rounded-xl shadow-xl border border-gray-100 z-50 overflow-hidden
animate-in fade-in slide-in-from-top-2 duration-200">
                          <button
                            onClick={() => handleToggleField(u.id,
'is_verified', u.is_verified)}
                            className="w-full px-4 py-3 text-left text-sm
hover:bg-gray-50 flex items-center gap-2"
                          >
                            {u.is_verified ? <ShieldOff size={16} /> :
<Shield size={16} />}
                            {u.is_verified ? 'Unverify User' : 'Verify
User'}
                          </button>
                          <button
                            onClick={() => handleToggleField(u.id,
'is_active', u.is_active)}
                            className="w-full px-4 py-3 text-left text-sm
hover:bg-gray-50 flex items-center gap-2"
                          >
                            {u.is_active ? <UserX size={16} /> :
<UserCheck size={16} />}
                            {u.is_active ? 'Deactivate User' : 'Activate
User'}
                          </button>
                          <button
                            onClick={() => handleToggleField(u.id,
'is_premium', u.is_premium)}
```

```
                          className="w-full px-4 py-3 text-left text-sm
hover:bg-gray-50 flex items-center gap-2"
                        >
                          {u.is_premium ? <StarOff size={16} /> : <Star
size={16} />}
                          {u.is_premium ? 'Remove Premium' : 'Make
Premium'}
                        </button>
                        <Link
                          to={`/admin/users/${u.id}`}
                          className="w-full px-4 py-3 text-left text-sm
hover:bg-gray-50 flex items-center gap-2 text-primary font-bold border-t
border-gray-50"
                        >
                          <ExternalLink size={16} />
                          View Details
                        </Link>
                      </div>
                    )}
                  </td>
                </tr>
              ))
            ) : (
              <tr>
                <td colSpan={8} className="px-6 py-12 text-center
text-gray-400">
                  No users found matching filters
                </td>
              </tr>
            )}
          </tbody>
        </table>
      </div>

      {/* Pagination */}
      {totalPages > 1 && (
        <div className="px-6 py-4 bg-gray-50 flex items-center
justify-between border-t border-gray-100">
          <p className="text-sm text-gray-500">
```

```
              Showing <span className="font-bold text-gray-900">{(page -
1) * limit + 1}</span> to <span className="font-bold
text-gray-900">{Math.min(page * limit, totalCount)}</span> of <span
className="font-bold text-gray-900">{totalCount}</span> users
            </p>
            <div className="flex items-center gap-2">
              <Button
                variant="outline"
                size="sm"
                disabled={page === 1}
                onClick={() => setPage(page - 1)}
              >
                <ChevronLeft size={16} />
              </Button>
              <div className="flex items-center gap-1">
                {[...Array(totalPages)].map((_, i) => (
                  <button
                    key={i}
                    onClick={() => setPage(i + 1)}
                    className={`w-8 h-8 rounded-lg text-xs font-bold
transition-all ${
                      page === i + 1 ? 'bg-primary text-white shadow-md' :
'hover:bg-gray-200 text-gray-600'
                    }`}
                  >
                    {i + 1}
                  </button>
                ))}
              </div>
              <Button
                variant="outline"
                size="sm"
                disabled={page === totalPages}
                onClick={() => setPage(page + 1)}
              >
                <ChevronRight size={16} />
              </Button>
            </div>
          </div>
        )}
```

```
      </Card>

      {/* Add Profile Modal */}
      {showAddModal && (
        <div className="fixed inset-0 bg-black/50 z-50 flex items-center
justify-center p-4">
          <div className="bg-white rounded-2xl max-w-2xl w-full p-6
shadow-2xl max-h-[90vh] overflow-y-auto">
            <div className="flex justify-between items-center mb-6">
              <h2 className="text-2xl font-bold text-gray-900">Add New
Profile</h2>
              <button onClick={() => setShowAddModal(false)}
className="p-2 hover:bg-gray-100 rounded-full transition-colors">
                <X size={20} />
              </button>
            </div>

            <form onSubmit={handleAddProfile} className="space-y-4">
              <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
                <div>
                  <label className="block text-sm font-medium
text-gray-700 mb-1">First Name *</label>
                  <Input
                    required
                    value={newProfile.first_name}
                    onChange={(e) => setNewProfile({...newProfile,
first_name: e.target.value})}
                  />
                </div>
                <div>
                  <label className="block text-sm font-medium
text-gray-700 mb-1">Last Name *</label>
                  <Input
                    required
                    value={newProfile.last_name}
                    onChange={(e) => setNewProfile({...newProfile,
last_name: e.target.value})}
                  />
                </div>
                <div>
```

```
                  <label className="block text-sm font-medium
text-gray-700 mb-1">Gender *</label>
                  <Select
                    required
                    value={newProfile.gender}
                    onChange={(e) => setNewProfile({...newProfile, gender:
e.target.value})}
                    options={[
                      { value: 'male', label: 'Male' },
                      { value: 'female', label: 'Female' }
                    ]}
                  />
                </div>
                <div>
                  <label className="block text-sm font-medium
text-gray-700 mb-1">Date of Birth *</label>
                  <Input
                    type="date"
                    required
                    value={newProfile.date_of_birth}
                    onChange={(e) => setNewProfile({...newProfile,
date_of_birth: e.target.value})}
                  />
                </div>
                <div>
                  <label className="block text-sm font-medium
text-gray-700 mb-1">Profile For</label>
                  <Select
                    value={newProfile.profile_for}
                    onChange={(e) => setNewProfile({...newProfile,
profile_for: e.target.value})}
                    options={[
                      { value: 'myself', label: 'Myself' },
                      { value: 'son', label: 'Son' },
                      { value: 'daughter', label: 'Daughter' },
                      { value: 'brother', label: 'Brother' },
                      { value: 'sister', label: 'Sister' },
                      { value: 'relative', label: 'Relative' },
                      { value: 'friend', label: 'Friend' }
                    ]}
```

```
                />
              </div>
              <div>
                <label className="block text-sm font-medium
text-gray-700 mb-1">Phone Number</label>
                <Input
                  type="tel"
                  value={newProfile.phone}
                  onChange={(e) => setNewProfile({...newProfile, phone:
e.target.value})}
                />
              </div>
              <div>
                <label className="block text-sm font-medium
text-gray-700 mb-1">Religion</label>
                <Input
                  value={newProfile.religion}
                  onChange={(e) => setNewProfile({...newProfile,
religion: e.target.value})}
                />
              </div>
              <div>
                <label className="block text-sm font-medium
text-gray-700 mb-1">Caste</label>
                <Input
                  value={newProfile.caste}
                  onChange={(e) => setNewProfile({...newProfile, caste:
e.target.value})}
                />
              </div>
            </div>

            <div className="flex gap-3 mt-8 pt-4 border-t
border-gray-100">
              <Button type="button" variant="outline" onClick={() =>
setShowAddModal(false)} className="flex-1">
                Cancel
              </Button>
              <Button type="submit" disabled={addingProfile}
className="flex-1">
```

```
                  {addingProfile ? 'Adding...' : 'Add Profile'}
                </Button>
              </div>
            </form>
          </div>
        </div>
      )}
    </div>
  );
}
```

24.src/pages/admin/AdminVerificationPage.tsx

```
import React, { useState, useEffect } from 'react';
import { Link } from 'react-router-dom';
import { Shield, CheckCircle2, XCircle, Clock, Search, Filter, Eye,
Download, AlertTriangle } from 'lucide-react';
import toast from 'react-hot-toast';
import { getPendingVerifications, approveDocument, rejectDocument } from
'../../lib/actions/documentActions';
import { VerificationDocument, Profile } from '../../lib/types';
import { useAuthStore } from '../../store/authStore';
import Button from '../../components/ui/Button';
import Badge from '../../components/ui/Badge';
import Spinner from '../../components/ui/Spinner';
import Avatar from '../../components/ui/Avatar';

interface PendingVerification extends VerificationDocument {
  profile?: Profile;
}

export default function AdminVerificationPage() {
  const { user } = useAuthStore();
  const [documents, setDocuments] = useState<PendingVerification[]>([]);
  const [loading, setLoading] = useState(true);
  const [filter, setFilter] = useState<'all' | 'aadhaar_front' |
'aadhaar_back' | 'biodata'>('all');
  const [searchTerm, setSearchTerm] = useState('');
```

```
  // Modal state
  const [selectedDoc, setSelectedDoc] = useState<PendingVerification |
null>(null);
  const [rejectReason, setRejectReason] = useState('');
  const [actionLoading, setActionLoading] = useState(false);

  useEffect(() => {
    fetchPendingDocs();
  }, []);

  const fetchPendingDocs = async () => {
    try {
      setLoading(true);
      const docs = await getPendingVerifications();
      setDocuments(docs);
    } catch (error) {
      console.error('Error fetching pending verifications:', error);
      toast.error('Failed to load pending verifications');
    } finally {
      setLoading(false);
    }
  };

  const handleApprove = async (docId: string, userId: string) => {
    try {
      setActionLoading(true);
      await approveDocument(docId, userId);
      toast.success('Document approved successfully');
      setSelectedDoc(null);
      fetchPendingDocs(); // Refresh list
    } catch (error: any) {
      console.error('Error approving document:', error);
      toast.error(error.message || 'Failed to approve document');
    } finally {
      setActionLoading(false);
    }
  };

  const handleReject = async (docId: string) => {
    if (!rejectReason.trim()) {
```

```
      toast.error('Please provide a reason for rejection');
      return;
    }

    try {
      setActionLoading(true);
      await rejectDocument(docId, user!.id, rejectReason);
      toast.success('Document rejected');
      setSelectedDoc(null);
      setRejectReason('');
      fetchPendingDocs(); // Refresh list
    } catch (error: any) {
      console.error('Error rejecting document:', error);
      toast.error(error.message || 'Failed to reject document');
    } finally {
      setActionLoading(false);
    }
  };

  const filteredDocs = documents.filter(doc => {
    const matchesFilter = filter === 'all' || doc.document_type ===
filter;
    const matchesSearch = searchTerm === '' ||

doc.profile?.first_name?.toLowerCase().includes(searchTerm.toLowerCase())
||

doc.profile?.last_name?.toLowerCase().includes(searchTerm.toLowerCase())
||

doc.profile?.profile_id?.toLowerCase().includes(searchTerm.toLowerCase());

    return matchesFilter && matchesSearch;
  });

  const getDocTypeLabel = (type: string) => {
    switch(type) {
      case 'aadhaar_front': return 'Aadhaar (Front)';
      case 'aadhaar_back': return 'Aadhaar (Back)';
      case 'biodata': return 'Biodata PDF';
```

```
      default: return type;
    }
  };

  return (
    <div className="min-h-screen bg-gray-50 py-8">
      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
        <div className="flex flex-col md:flex-row md:items-center
justify-between mb-8 gap-4">
          <div>
            <h1 className="text-3xl font-bold text-gray-900 flex
items-center gap-3">
              <Shield className="text-primary" size={32} />
              Document Verification
            </h1>
            <p className="text-gray-500 mt-1">Review and approve user
uploaded documents</p>
          </div>

          <div className="flex items-center gap-3 bg-white p-2 rounded-lg
shadow-sm border border-gray-200">
            <div className="px-4 py-2 bg-yellow-50 text-yellow-800
rounded-md flex items-center gap-2 font-medium">
              <Clock size={18} />
              {documents.length} Pending
            </div>
          </div>
        </div>

        {/* Filters and Search */}
        <div className="bg-white p-4 rounded-xl shadow-sm border
border-gray-200 mb-6 flex flex-col sm:flex-row gap-4 justify-between">
          <div className="flex items-center gap-2 overflow-x-auto pb-2
sm:pb-0 hide-scrollbar">
            <Filter size={18} className="text-gray-400 shrink-0" />
            <button
              onClick={() => setFilter('all')}
              className={`px-3 py-1.5 rounded-full text-sm font-medium
whitespace-nowrap transition-colors ${filter === 'all' ? 'bg-primary
text-white' : 'bg-gray-100 text-gray-600 hover:bg-gray-200'}`}
```

```
            >
              All Documents
            </button>
            <button
              onClick={() => setFilter('aadhaar_front')}
              className={`px-3 py-1.5 rounded-full text-sm font-medium
whitespace-nowrap transition-colors ${filter === 'aadhaar_front' ?
'bg-primary text-white' : 'bg-gray-100 text-gray-600 hover:bg-gray-200'}`}
            >
              Aadhaar Front
            </button>
            <button
              onClick={() => setFilter('aadhaar_back')}
              className={`px-3 py-1.5 rounded-full text-sm font-medium
whitespace-nowrap transition-colors ${filter === 'aadhaar_back' ?
'bg-primary text-white' : 'bg-gray-100 text-gray-600 hover:bg-gray-200'}`}
            >
              Aadhaar Back
            </button>
            <button
              onClick={() => setFilter('biodata')}
              className={`px-3 py-1.5 rounded-full text-sm font-medium
whitespace-nowrap transition-colors ${filter === 'biodata' ? 'bg-primary
text-white' : 'bg-gray-100 text-gray-600 hover:bg-gray-200'}`}
            >
              Biodata
            </button>
          </div>

          <div className="relative w-full sm:w-64 shrink-0">
            <div className="absolute inset-y-0 left-0 pl-3 flex
items-center pointer-events-none">
              <Search size={18} className="text-gray-400" />
            </div>
            <input
              type="text"
              placeholder="Search by name or ID..."
              value={searchTerm}
              onChange={(e) => setSearchTerm(e.target.value)}
```

```
              className="block w-full pl-10 pr-3 py-2 border
border-gray-300 rounded-lg leading-5 bg-white placeholder-gray-500
focus:outline-none focus:ring-primary focus:border-primary sm:text-sm"
            />
          </div>
        </div>

        {/* Document List */}
        {loading ? (
          <div className="py-20 flex justify-center">
            <Spinner size="lg" />
          </div>
        ) : filteredDocs.length === 0 ? (
          <div className="bg-white rounded-xl shadow-sm border
border-gray-200 p-12 text-center">
            <Shield className="mx-auto h-16 w-16 text-gray-300 mb-4" />
            <h3 className="text-xl font-medium text-gray-900 mb-2">No
pending documents</h3>
            <p className="text-gray-500">All caught up! There are no
documents waiting for verification.</p>
          </div>
        ) : (
          <div className="bg-white rounded-xl shadow-sm border
border-gray-200 overflow-hidden">
            <div className="overflow-x-auto">
              <table className="min-w-full divide-y divide-gray-200">
                <thead className="bg-gray-50">
                  <tr>
                    <th scope="col" className="px-6 py-3 text-left text-xs
font-medium text-gray-500 uppercase tracking-wider">
                      User
                    </th>
                    <th scope="col" className="px-6 py-3 text-left text-xs
font-medium text-gray-500 uppercase tracking-wider">
                      Document Type
                    </th>
                    <th scope="col" className="px-6 py-3 text-left text-xs
font-medium text-gray-500 uppercase tracking-wider">
                      Uploaded Date
                    </th>
```

```
                    <th scope="col" className="px-6 py-3 text-right
text-xs font-medium text-gray-500 uppercase tracking-wider">
                      Action
                    </th>
                  </tr>
                </thead>
                <tbody className="bg-white divide-y divide-gray-200">
                  {filteredDocs.map((doc) => (
                    <tr key={doc.id} className="hover:bg-gray-50
transition-colors">
                      <td className="px-6 py-4 whitespace-nowrap">
                        <div className="flex items-center">
                          <div className="flex-shrink-0 h-10 w-10">
                            <Avatar
                              src={doc.profile?.profile_photo_url}
                              fallbackName={doc.profile?.first_name ||
'User'}
                              size="sm"
                            />
                          </div>
                          <div className="ml-4">
                            <div className="text-sm font-medium
text-gray-900">
                              {doc.profile?.first_name}
{doc.profile?.last_name}
                            </div>
                            <div className="text-sm text-gray-500">
                              ID: {doc.profile?.profile_id ||
doc.user_id.substring(0, 8)}
                            </div>
                          </div>
                        </div>
                      </td>
                      <td className="px-6 py-4 whitespace-nowrap">
                        <div className="flex items-center gap-2">
                          <Badge variant="pending"
text={getDocTypeLabel(doc.document_type)} />
                        </div>
                      </td>
```

```
                    <td className="px-6 py-4 whitespace-nowrap text-sm
text-gray-500">
                      {new Date(doc.uploaded_at).toLocaleDateString()}
at {new Date(doc.uploaded_at).toLocaleTimeString([], {hour: '2-digit',
minute:'2-digit'})}
                    </td>
                    <td className="px-6 py-4 whitespace-nowrap
text-right text-sm font-medium">
                      <Button
                        variant="outline"
                        size="sm"
                        onClick={() => setSelectedDoc(doc)}
                        className="flex items-center gap-1 ml-auto"
                      >
                        <Eye size={16} /> Review
                      </Button>
                    </td>
                  </tr>
                ))}
              </tbody>
            </table>
          </div>
        </div>
      )}
    </div>

    {/* Review Modal */}
    {selectedDoc && (
      <div className="fixed inset-0 z-50 overflow-y-auto"
aria-labelledby="modal-title" role="dialog" aria-modal="true">
        <div className="flex items-end justify-center min-h-screen pt-4
px-4 pb-20 text-center sm:block sm:p-0">
          <div className="fixed inset-0 bg-gray-500 bg-opacity-75
transition-opacity" aria-hidden="true" onClick={() => !actionLoading &&
setSelectedDoc(null)}></div>

          <span className="hidden sm:inline-block sm:align-middle
sm:h-screen" aria-hidden="true">&#8203;</span>
```

```
            <div className="inline-block align-bottom bg-white rounded-2xl
text-left overflow-hidden shadow-xl transform transition-all sm:my-8
sm:align-middle sm:max-w-4xl sm:w-full">
              <div className="bg-white px-4 pt-5 pb-4 sm:p-6 sm:pb-4">
                <div className="sm:flex sm:items-start">
                  <div className="mt-3 text-center sm:mt-0 sm:ml-4
sm:text-left w-full">
                    <div className="flex justify-between items-center mb-4
pb-4 border-b border-gray-100">
                      <h3 className="text-xl leading-6 font-bold
text-gray-900" id="modal-title">
                        Review Document:
{getDocTypeLabel(selectedDoc.document_type)}
                      </h3>
                      <button
                        onClick={() => setSelectedDoc(null)}
                        className="text-gray-400 hover:text-gray-500
focus:outline-none"
                        disabled={actionLoading}
                      >
                        <span className="sr-only">Close</span>
                        <XCircle size={24} />
                      </button>
                    </div>

                    <div className="grid grid-cols-1 md:grid-cols-3
gap-6">
                      {/* User Info Sidebar */}
                      <div className="md:col-span-1 bg-gray-50 p-4
rounded-xl border border-gray-200">
                        <h4 className="font-medium text-gray-900 mb-4
border-b border-gray-200 pb-2">User Details</h4>
                        <div className="flex items-center gap-3 mb-4">
                          <Avatar
                            src={selectedDoc.profile?.profile_photo_url}
                            fallbackName={selectedDoc.profile?.first_name
|| 'User'}
                            size="md"
                          />
                          <div>
```

```
                            <p className="font-medium
text-gray-900">{selectedDoc.profile?.first_name}
{selectedDoc.profile?.last_name}</p>
                            <p className="text-xs
text-gray-500">{selectedDoc.profile?.profile_id}</p>
                          </div>
                        </div>

                        <div className="space-y-2 text-sm">
                          <p><span
className="text-gray-500">Gender:</span> {selectedDoc.profile?.gender}</p>
                          <p><span className="text-gray-500">DOB:</span>
{selectedDoc.profile?.date_of_birth ? new
Date(selectedDoc.profile.date_of_birth).toLocaleDateString() : 'N/A'}</p>
                          <p><span
className="text-gray-500">Uploaded:</span> {new
Date(selectedDoc.uploaded_at).toLocaleDateString()}</p>
                        </div>

                        <div className="mt-6 pt-4 border-t
border-gray-200">
                          <a
                            href={selectedDoc.file_url}
                            target="_blank"
                            rel="noopener noreferrer"
                            className="flex items-center justify-center
gap-2 w-full py-2 px-4 border border-gray-300 rounded-md shadow-sm text-sm
font-medium text-gray-700 bg-white hover:bg-gray-50 focus:outline-none
focus:ring-2 focus:ring-offset-2 focus:ring-primary"
                          >
                            <Download size={16} /> Download File
                          </a>
                        </div>
                      </div>

                      {/* Document Viewer */}
                      <div className="md:col-span-2 flex flex-col">
                        <div className="bg-gray-100 rounded-xl border
border-gray-200 overflow-hidden flex-1 min-h-[400px] flex items-center
justify-center relative">
```

```
                        {selectedDoc.file_type.startsWith('image/') ? (
                          <img
                            src={selectedDoc.file_url}
                            alt="Verification Document"
                            className="max-w-full max-h-[500px]
object-contain"
                          />
                        ) : selectedDoc.file_type === 'application/pdf'
? (
                          <div className="text-center p-8">
                            <Shield className="mx-auto h-16 w-16
text-gray-400 mb-4" />
                            <p className="text-gray-600 mb-4">PDF
Document Viewer</p>
                            <a
                              href={selectedDoc.file_url}
                              target="_blank"
                              rel="noopener noreferrer"
                              className="inline-flex items-center gap-2
px-4 py-2 border border-transparent text-sm font-medium rounded-md
shadow-sm text-white bg-primary hover:bg-primary-700"
                            >
                              <Eye size={16} /> Open PDF in New Tab
                            </a>
                          </div>
                        ) : (
                          <div className="text-center p-8">
                            <AlertTriangle className="mx-auto h-16 w-16
text-yellow-500 mb-4" />
                            <p className="text-gray-600">Preview not
available for this file type.</p>
                          </div>
                        )}
                      </div>

                      {/* Rejection Reason Input */}
                      <div className="mt-4">
                        <label htmlFor="rejectReason" className="block
text-sm font-medium text-gray-700 mb-1">
                          Rejection Reason (Required if rejecting)
```

```
                        </label>
                        <textarea
                          id="rejectReason"
                          rows={2}
                          className="shadow-sm focus:ring-red-500
focus:border-red-500 block w-full sm:text-sm border-gray-300 rounded-md"
                          placeholder="e.g., Image is blurry, name
doesn't match profile, etc."
                          value={rejectReason}
                          onChange={(e) =>
setRejectReason(e.target.value)}
                        />
                      </div>
                    </div>
                  </div>
                </div>
              </div>

              <div className="bg-gray-50 px-4 py-3 sm:px-6 sm:flex
sm:flex-row-reverse gap-3 border-t border-gray-200">
                <Button
                  variant="primary"
                  onClick={() => handleApprove(selectedDoc.id,
selectedDoc.user_id)}
                  loading={actionLoading}
                  className="w-full sm:w-auto flex items-center
justify-center gap-2 bg-green-600 hover:bg-green-700"
                >
                  <CheckCircle2 size={18} /> Approve Document
                </Button>
                <Button
                  variant="danger"
                  onClick={() => handleReject(selectedDoc.id)}
                  loading={actionLoading}
                  className="w-full sm:w-auto mt-3 sm:mt-0 flex
items-center justify-center gap-2"
                >
                  <XCircle size={18} /> Reject
                </Button>
```

```
                <Button
                  variant="ghost"
                  onClick={() => setSelectedDoc(null)}
                  disabled={actionLoading}
                  className="w-full sm:w-auto mt-3 sm:mt-0"
                >
                  Cancel
                </Button>
              </div>
            </div>
          </div>
        </div>
      )}
    </div>
  );
}
```

Priority 7 — Business Logic:

25.src/lib/actions/authActions.ts

```
export async function registerUser(data: {
  email: string
  password: string
  first_name: string
  last_name: string
  gender: string
  profile_for: string
  date_of_birth: string
  phone?: string
}) {
  const response = await fetch('/api/auth/register', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(data)
  })

  if (!response.ok) {
```

```
    const err = await response.json()
    throw new Error(err.error || 'Registration failed')
  }

  const authData = await response.json()
  if (authData.token) {
    localStorage.setItem('atmilan-token', authData.token)
  }
  return authData
}

export async function loginUser(email: string, password: string) {
  const response = await fetch('/api/auth/login', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ email, password })
  })

  if (!response.ok) {
    const err = await response.json()
    throw new Error(err.error || 'Login failed')
  }

  const data = await response.json()
  if (data.token) {
    localStorage.setItem('atmilan-token', data.token)
  }
  return data
}

export async function uploadDocument(
  userId: string,
  file: File,
  documentType: 'aadhaar_front' | 'aadhaar_back' | 'biodata'
) {
  const formData = new FormData()
  formData.append('file', file)
  formData.append('userId', userId)
  formData.append('documentType', documentType)
```

```
  const response = await fetch('/api/upload', {
    method: 'POST',
    body: formData
  })

  if (!response.ok) {
    const err = await response.json()
    throw new Error(err.error || 'Upload failed')
  }

  return response.json()
}

export async function getUserDocuments(userId: string) {
  const response = await fetch(`/api/documents/${userId}`)
  if (!response.ok) throw new Error('Failed to fetch documents')
  return response.json()
}

export async function updatePassword(newPassword: string) {
  // Local placeholder
  console.log('Update password locally:', newPassword)
}

export async function resetPassword(email: string) {
  // Local placeholder
  console.log('Reset password locally for:', email)
}

export async function updateProfileField(userId: string, updates:
Record<string, any>) {
  const response = await fetch(`/api/profiles/${userId}/personal`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(updates)
  })
  if (!response.ok) throw new Error('Failed to update profile field')
}

export async function deactivateAccount(userId: string) {
```

```
  const response = await fetch(`/api/profiles/${userId}/personal`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ is_active: false })
  })
  if (!response.ok) throw new Error('Failed to deactivate account')
  localStorage.removeItem('atmilan-token')
}

export async function deleteAccount(userId: string) {
  // Local implementation: just deactivate for now
  const response = await fetch(`/api/profiles/${userId}/personal`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ is_active: false })
  })
  localStorage.removeItem('atmilan-token')
}
```

26.src/lib/actions/creditActions.ts (or similar)

We not build yet

27. src/lib/actions/interestActions.ts

```
// Get received interests (people who sent interest to me)
export async function getReceivedInterests(userId: string) {
  const response = await fetch(`/api/interests/received/${userId}`);
  if (!response.ok) throw new Error('Failed to fetch received interests');
  return await response.json();
}

// Get sent interests
export async function getSentInterests(userId: string) {
  const response = await fetch(`/api/interests/sent/${userId}`);
  if (!response.ok) throw new Error('Failed to fetch sent interests');
  return await response.json();
}
```

```
// Get accepted interests (for chat)
export async function getAcceptedInterests(userId: string) {
  // Local implementation: filter sent and received interests for
'accepted' status
  const [received, sent] = await Promise.all([
    getReceivedInterests(userId),
    getSentInterests(userId)
  ]);

  const acceptedReceived = received.filter((i: any) => i.status ===
'accepted');
  const acceptedSent = sent.filter((i: any) => i.status === 'accepted');

  return [...acceptedReceived, ...acceptedSent];
}

// Get rejected/declined interests
export async function getRejectedInterests(userId: string) {
  const [received, sent] = await Promise.all([
    getReceivedInterests(userId),
    getSentInterests(userId)
  ]);

  const declinedReceived = received.filter((i: any) => i.status ===
'declined');
  const declinedSent = sent.filter((i: any) => i.status === 'declined');

  return [...declinedReceived, ...declinedSent];
}

// Accept interest
export async function acceptInterest(interestId: string, senderId: string,
receiverName: string) {
  const response = await fetch(`/api/interests/${interestId}/status`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ status: 'accepted', receiverName })
  });
  if (!response.ok) throw new Error('Failed to accept interest');
}
```

```
// Decline interest
export async function declineInterest(interestId: string, senderId:
string, receiverName: string) {
  const response = await fetch(`/api/interests/${interestId}/status`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ status: 'declined', receiverName })
  });
  if (!response.ok) throw new Error('Failed to decline interest');
}

// Cancel sent interest
export async function cancelInterest(interestId: string) {
  // Local implementation: we need a delete endpoint or status update
  const response = await fetch(`/api/interests/${interestId}`, {
    method: 'DELETE'
  });
  if (!response.ok) throw new Error('Failed to cancel interest');
}

// Get interest counts
export async function getInterestCounts(userId: string) {
  const [received, sent] = await Promise.all([
    getReceivedInterests(userId),
    getSentInterests(userId)
  ]);

  return {
    received: received.filter((i: any) => i.status === 'pending').length,
    sent: sent.length,
    accepted: [...received, ...sent].filter((i: any) => i.status ===
'accepted').length,
    rejected: [...received, ...sent].filter((i: any) => i.status ===
'declined').length
  }
}

export async function sendInterest(senderId: string, receiverId: string) {
  const response = await fetch('/api/interests', {
```

```
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ sender_id: senderId, receiver_id: receiverId })
  });
  if (!response.ok) throw new Error('Failed to send interest');
  return await response.json();
}
```

28.src/lib/actions/messageActions.ts

```
// Check if two users can chat (mutual accepted interest)
export async function canChat(userId: string, otherUserId: string) {
  const response = await fetch(`/api/interests/received/${userId}`);
  if (!response.ok) return false;
  const received = await response.json();

  const sentResponse = await fetch(`/api/interests/sent/${userId}`);
  if (!sentResponse.ok) return false;
  const sent = await sentResponse.json();

  const mutual = [...received, ...sent].find((i: any) =>
    i.status === 'accepted' &&
    ((i.sender_id === userId && i.receiver_id === otherUserId) ||
     (i.sender_id === otherUserId && i.receiver_id === userId))
  );

  return !!mutual;
}

// Get conversations list (accepted interests with last message)
export async function getConversations(userId: string) {
  const response = await fetch(`/api/interests/received/${userId}`);
  if (!response.ok) return [];
  const received = await response.json();

  const sentResponse = await fetch(`/api/interests/sent/${userId}`);
  if (!sentResponse.ok) return [];
  const sent = await sentResponse.json();
```

```
  const acceptedInterests = [...received, ...sent].filter((i: any) =>
i.status === 'accepted');

  if (acceptedInterests.length === 0) return []

  const conversations = await Promise.all(
    acceptedInterests.map(async (interest) => {
      const otherUser = interest.sender_id === userId ? interest.receiver
: interest.sender
      const otherUserId = interest.sender_id === userId ?
interest.receiver_id : interest.sender_id

      const msgRes = await
fetch(`/api/messages/${userId}/${otherUserId}`);
      const messages = msgRes.ok ? await msgRes.json() : [];
      const lastMsg = messages.length > 0 ? messages[messages.length - 1]
: null;

      const unreadCount = messages.filter((m: any) => m.receiver_id ===
userId && !m.is_read).length;

      return {
        interestId: interest.id,
        otherUser,
        otherUserId,
        lastMessage: lastMsg,
        unreadCount
      }
    })
  )

  conversations.sort((a, b) => {
    if (!a.lastMessage && !b.lastMessage) return 0
    if (!a.lastMessage) return 1
    if (!b.lastMessage) return -1
    return new Date(b.lastMessage.created_at).getTime() - new
Date(a.lastMessage.created_at).getTime()
  })

  return conversations
```

```
}

// Get messages between two users
export async function getMessages(userId: string, otherUserId: string,
limit: number = 50) {
  const response = await fetch(`/api/messages/${userId}/${otherUserId}`);
  if (!response.ok) throw new Error('Failed to fetch messages');
  const messages = await response.json();
  return messages.slice(-limit);
}

// Send message
export async function sendMessage(senderId: string, receiverId: string,
content: string) {
  const response = await fetch('/api/messages', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ sender_id: senderId, receiver_id: receiverId,
content: content.trim() })
  });
  if (!response.ok) throw new Error('Failed to send message');
  return await response.json();
}

// Mark messages as read
export async function markMessagesAsRead(userId: string, otherUserId:
string) {
  try {
    await fetch(`/api/messages/${userId}/${otherUserId}/read`, { method:
'POST' });
  } catch (error) {
    console.error('Error marking messages as read:', error);
  }
}

// Get total unread message count
export async function getUnreadMessageCount(userId: string) {
  try {
    const response = await fetch(`/api/messages/unread-count/${userId}`);
    if (!response.ok) return 0;
```

```
    const data = await response.json();
    return data.count;
  } catch (error) {
    console.error('Error getting unread count:', error);
    return 0;
  }
}

// Get unread notification count
export async function getUnreadNotificationCount(userId: string) {
  const response = await fetch(`/api/notifications/${userId}`);
  if (!response.ok) return 0;
  const notifications = await response.json();
  return notifications.filter((n: any) => !n.is_read).length;
}

// Get notifications
export async function getNotifications(userId: string, limit: number = 20)
{
  const response = await fetch(`/api/notifications/${userId}`);
  if (!response.ok) throw new Error('Failed to fetch notifications');
  const notifications = await response.json();
  return notifications.sort((a: any, b: any) => new
Date(b.created_at).getTime() - new Date(a.created_at).getTime()).slice(0,
limit);
}

// Mark notification as read
export async function markNotificationRead(notificationId: string) {
  await fetch(`/api/notifications/${notificationId}/read`, { method:
'POST' });
}

// Mark all notifications as read
export async function markAllNotificationsRead(userId: string) {
  try {
    await fetch(`/api/notifications/${userId}/read-all`, { method: 'POST'
});
  } catch (error) {
    console.error('Error marking all notifications as read:', error);
```

```
  }
}
```

29.src/lib/constants.ts

```
export const DEFAULT_MALE_AVATAR =
'https://img.freepik.com/premium-vector/man-avatar-profile-picture-vector-
illustration_268834-541.jpg'
export const DEFAULT_FEMALE_AVATAR =
'https://www.uiu.ac.bd/wp-content/uploads/2025/10/female-300n300.jpg'

export function getDefaultAvatar(gender: string): string {
  if (gender === 'Female') return DEFAULT_FEMALE_AVATAR
  return DEFAULT_MALE_AVATAR
}

export const profileForOptions = ['Self', 'Son', 'Daughter', 'Brother',
'Sister', 'Friend', 'Relative'];
export const genderOptions = ['Male', 'Female'];
export const maritalStatusOptions = ['Never Married', 'Divorced',
'Widowed', 'Awaiting Divorce', 'Annulled'];
export const religionOptions = ['Hindu', 'Muslim', 'Christian', 'Sikh',
'Buddhist', 'Jain', 'Parsi', 'Other'];

export const castesByReligion: Record<string, string[]> = {
  Hindu: ['Brahmin', 'Kshatriya', 'Vaishya', 'Kayastha', 'Rajput',
'Maratha', 'Jat', 'Patel', 'Yadav', 'Gupta', 'Agarwal', 'Baniya', 'Nair',
'Reddy', 'Naidu', 'Iyer', 'Iyengar', 'Sharma', 'Verma', 'Other'],
  Muslim: ['Sunni', 'Shia', 'Pathan', 'Syed', 'Sheikh', 'Ansari',
'Qureshi', 'Other'],
  Christian: ['Catholic', 'Protestant', 'Orthodox', 'Born Again',
'Other'],
  Sikh: ['Jat', 'Khatri', 'Arora', 'Ramgharia', 'Saini', 'Other'],
  Buddhist: ['Mahayana', 'Theravada', 'Vajrayana', 'Other'],
  Jain: ['Digambar', 'Shwetambar', 'Other'],
  Parsi: ['Parsi', 'Irani', 'Other'],
  Other: ['Other']
};
```

```
export const motherTongueOptions = ['Hindi', 'English', 'Bengali',
'Telugu', 'Marathi', 'Tamil', 'Urdu', 'Gujarati', 'Kannada', 'Odia',
'Malayalam', 'Punjabi', 'Assamese', 'Maithili', 'Sindhi', 'Konkani',
'Dogri', 'Kashmiri', 'Sanskrit', 'Other'];

export const heightOptions = Array.from({ length: 37 }, (_, i) => {
  const totalInches = 48 + i; // 4'0" is 48 inches
  const feet = Math.floor(totalInches / 12);
  const inches = totalInches % 12;
  const cm = Math.round(totalInches * 2.54);
  return { label: `${feet}' ${inches}" (${cm} cm)`, value: cm };
});

export const educationOptions = ['Below 10th', '10th', '12th', 'Diploma',
'Bachelors', 'Masters (MBA/MS/MA)', 'PhD/Doctorate', 'B.Tech/B.E',
'MBBS/BDS', 'BCA/MCA', 'B.Com/M.Com', 'BA/MA', 'BSc/MSc', 'LLB/LLM',
'CA/CS/ICWA', 'B.Pharm/M.Pharm', 'BBA/MBA', 'Other'];
export const occupationOptions = ['Private Job', 'Government Job',
'Business/Self Employed', 'Doctor', 'Engineer', 'Software Professional',
'Teacher/Professor', 'Lawyer/Legal', 'CA/CS', 'Banking/Finance', 'Civil
Services (IAS/IPS)', 'Defence/Armed Forces', 'Scientist/Researcher',
'Architect', 'Pilot', 'Farmer/Agriculture', 'Media/Journalism',
'Artist/Designer', 'Not Working', 'Student', 'Other'];
export const incomeOptions = ['Not Specified', 'Below 2 LPA', '2-4 LPA',
'4-6 LPA', '6-8 LPA', '8-10 LPA', '10-15 LPA', '15-20 LPA', '20-30 LPA',
'30-50 LPA', '50-75 LPA', '75 LPA - 1 Cr', '1 Cr+'];
export const bodyTypeOptions = ['Slim', 'Average', 'Athletic', 'Heavy'];
export const complexionOptions = ['Very Fair', 'Fair', 'Wheatish',
'Wheatish Brown', 'Dark'];
export const dietOptions = ['Vegetarian', 'Non-Vegetarian', 'Eggetarian',
'Jain', 'Vegan'];
export const smokingDrinkingOptions = ['No', 'Occasionally', 'Yes'];
export const familyTypeOptions = ['Joint', 'Nuclear', 'Other'];
export const familyStatusOptions = ['Middle Class', 'Upper Middle Class',
'Rich', 'Affluent'];
export const familyValuesOptions = ['Orthodox', 'Moderate', 'Liberal'];
export const manglikOptions = ['Yes', 'No', 'Not Sure', 'Not Applicable'];
export const bloodGroupOptions = ['A+', 'A-', 'B+', 'B-', 'AB+', 'AB-',
'O+', 'O-', 'Not Known'];
```

```
export const indianStates = [
  'Andhra Pradesh', 'Arunachal Pradesh', 'Assam', 'Bihar', 'Chhattisgarh',
'Goa', 'Gujarat', 'Haryana', 'Himachal Pradesh', 'Jharkhand', 'Karnataka',
'Kerala', 'Madhya Pradesh', 'Maharashtra', 'Manipur', 'Meghalaya',
'Mizoram', 'Nagaland', 'Odisha', 'Punjab', 'Rajasthan', 'Sikkim', 'Tamil
Nadu', 'Telangana', 'Tripura', 'Uttar Pradesh', 'Uttarakhand', 'West
Bengal',
  'Andaman and Nicobar Islands', 'Chandigarh', 'Dadra and Nagar Haveli and
Daman and Diu', 'Delhi', 'Jammu and Kashmir', 'Ladakh', 'Lakshadweep',
'Puducherry'
];

export const hobbiesList = ['Reading', 'Cooking', 'Traveling',
'Photography', 'Music', 'Dancing', 'Painting/Drawing', 'Gaming',
'Yoga/Meditation', 'Gym/Fitness', 'Gardening', 'Writing/Blogging',
'Movies/TV', 'Sports', 'Social Media', 'Volunteering', 'Crafts/DIY',
'Singing', 'Playing Instruments', 'Swimming'];

export const rashiOptions = ['Mesh', 'Vrishabh', 'Mithun', 'Kark',
'Singh', 'Kanya', 'Tula', 'Vrishchik', 'Dhanu', 'Makar', 'Kumbh', 'Meen'];

export const nakshatraOptions = [
  'Ashwini', 'Bharani', 'Krittika', 'Rohini', 'Mrigashirsha', 'Ardra',
'Punarvasu', 'Pushya', 'Ashlesha',
  'Magha', 'Purva Phalguni', 'Uttara Phalguni', 'Hasta', 'Chitra',
'Swati', 'Vishakha', 'Anuradha', 'Jyeshtha',
  'Mula', 'Purva Ashadha', 'Uttara Ashadha', 'Shravana', 'Dhanishta',
'Shatabhisha', 'Purva Bhadrapada', 'Uttara Bhadrapada', 'Revati'
];
```

30.src/lib/types.ts

```
export interface Profile {
  id: string
  profile_for: string
  first_name: string
  last_name: string
  gender: string
  date_of_birth: string
```

```
  marital_status: string
  religion: string
  caste: string
  sub_caste: string
  gotra: string
  mother_tongue: string
  height_cm: number
  weight_kg: number
  body_type: string
  complexion: string
  physical_disability: boolean
  blood_group: string
  about_me: string
  profile_photo_url: string
  profile_completion: number
  is_verified: boolean
  email_verified: boolean
  phone_verified: boolean
  aadhaar_verified: boolean
  photo_verified: boolean
  verification_status: string
  is_active: boolean
  is_premium: boolean
  premium_plan: string | null
  premium_end: string | null
  role: string
  profile_id: string
  created_at: string
  updated_at: string
}

export interface EducationCareer {
  id: string
  user_id: string
  highest_education: string
  education_field: string
  college_name: string
  occupation: string
  company_name: string
  designation: string
```

```
  annual_income: string
  working_city: string
  working_state: string
  working_country: string
}

export interface FamilyDetails {
  id: string
  user_id: string
  father_name: string
  father_occupation: string
  mother_name: string
  mother_occupation: string
  num_brothers: number
  brothers_married: number
  num_sisters: number
  sisters_married: number
  family_type: string
  family_status: string
  family_values: string
  native_place: string
  family_city: string
  family_state: string
  family_country: string
  about_family: string
}

export interface Lifestyle {
  id: string
  user_id: string
  diet: string
  smoking: string
  drinking: string
  hobbies: string[]
  interests: string[]
  languages_known: string[]
}

export interface HoroscopeDetails {
  id: string
```

```
  user_id: string
  manglik: string
  nakshatra: string
  rashi: string
  birth_time: string
  birth_place: string
}

export interface PartnerPreferences {
  id: string
  user_id: string
  age_from: number
  age_to: number
  height_from_cm: number
  height_to_cm: number
  marital_status_pref: string[]
  religion_pref: string[]
  caste_pref: string[]
  sub_caste_pref?: string[]
  mother_tongue_pref: string[]
  education_pref: string[]
  occupation_pref: string[]
  income_from: string
  income_to: string
  country_pref: string[]
  state_pref: string[]
  diet_pref: string[]
  smoking_pref: string
  drinking_pref: string
  manglik_pref: string
  about_partner: string
}

export interface Photo {
  id: string
  user_id: string
  photo_url: string
  is_profile_photo: boolean
  uploaded_at: string
}
```

```ts
export interface Interest {
  id: string
  sender_id: string
  receiver_id: string
  status: 'pending' | 'accepted' | 'declined'
  message: string
  created_at: string
  updated_at: string
  sender?: Profile
  receiver?: Profile
}

export interface Shortlist {
  id: string
  user_id: string
  shortlisted_user_id: string
  created_at: string
  shortlisted_user?: Profile
}

export interface ProfileView {
  id: string
  viewer_id: string
  viewed_id: string
  viewed_at: string
  viewer?: Profile
}

export interface Message {
  id: string
  sender_id: string
  receiver_id: string
  content: string
  is_read: boolean
  created_at: string
}

export interface Notification {
  id: string
```

```
  user_id: string
  type: string
  title: string
  body: string
  reference_id: string
  is_read: boolean
  created_at: string
}

export interface BlockReport {
  id: string
  reporter_id: string
  reported_id: string
  type: string
  reason: string
  status: string
  created_at: string
}

export interface SuccessStory {
  id: string
  user_id: string
  partner_name: string
  story_text: string
  photo_url: string
  is_approved: boolean
  created_at: string
  user?: Profile
}

export interface ContactMessage {
  id: string
  name: string
  email: string
  subject: string
  message: string
  is_resolved: boolean
  created_at: string
}
```

```typescript
export interface CompleteProfile extends Profile {
  education_career?: EducationCareer
  family_details?: FamilyDetails
  lifestyle?: Lifestyle
  horoscope_details?: HoroscopeDetails
  partner_preferences?: PartnerPreferences
  photos?: Photo[]
}

export interface VerificationDocument {
  id: string
  user_id: string
  document_type: 'aadhaar_front' | 'aadhaar_back' | 'biodata'
  file_url: string
  file_name: string
  file_type: string
  uploaded_at: string
  verification_status: 'pending' | 'approved' | 'rejected'
  admin_notes: string
  reviewed_by: string
  reviewed_at: string
}
```